

DAVID LONDOÑO

Pereira, Colombia (UTC-5) | +57 317 2939347 | dlondon.dev@gmail.com | linkedin.com/in/davidldv | github.com/davidldv | davidlondon.dev

AI Full-Stack Engineer. I ship production systems in TypeScript, Next.js, Node.js and PostgreSQL, and I work with coding agents from a written architecture and spec rather than from prompts. Two projects show how: **authzscan**, an agent pipeline that reviews existing Next.js codebases (I have inventoried repos of up to 3,000 files and 647 endpoints with it) and publishes its own false-positive rate, and **Ghost AI**, a collaborative canvas that turns a system design into a Markdown spec an implementation pipeline can run. English C1, Spanish native, German B1. Available full time, remote (LATAM/EU/US) or relocation.

Projects

Ghost AI | *Live* | *Code* | *Writeup*

Next.js | **Liveblocks** | **AI SDK**

Real-time architecture canvas: a team describes a system, an agent lays it out, the graph exports as a spec.

- Multiplayer canvas on **Liveblocks** and **React Flow**. Nodes and edges are rows in Postgres through **Prisma**, so the graph is data the app can query rather than a drawing. Prompts go through the **Vercel AI SDK** and come back as structured graph mutations; the model can only change the design through the same contract the UI uses. The finished graph exports as a **Markdown spec** (services, data ownership, integration points) that the next agent in the pipeline reads.
- Built the codebase **spec-first**. Four living context files (project overview, architecture and invariants, code standards, progress tracker) get read by every agent before it writes anything, and I scope and review each feature against them before implementation. When a refactor keeps the contracts those files document, I can merge it without re-reading the whole app.

authzscan | *Code* | *Writeup*

TypeScript | **ts-morph** | **SARIF**

Agents that review an existing Next.js codebase for authorization bugs, with published accuracy.

- Four phases. A deterministic **ts-morph** pass inventories every route handler, Server Action and auth idiom in the repo. A trace agent follows each client-controlled identifier to the query it reaches. A second, adversarial agent re-reads what trace found and kills its false positives. Results come out as **SARIF**, Markdown or JSON with exit codes CI can gate on. The model only runs where a judgment call is needed; everything that has a right answer stays deterministic.
- Measured on a seeded benchmark: **16/16** planted bugs found and **0/6** hardened twins flagged, at \$2.10 and 19 minutes per scan. An oracle runner checks that the harness scores correctly no matter how the model performs. I then ran it on a real open-source repo, kept 1 genuine bug out of 11 candidates, and fixed the seven tool bugs the benchmark had hidden in **v0.1.1**.

Materiales La Bodega | *Live Site*

Next.js | **PostgreSQL** | **Mercado Pago**

Solo-built e-commerce platform, live in production for a hardware retailer.

- The only engineer on a live storefront moving about **1.5M COP a day** (roughly USD 11k a month): requirements, architecture, the real-time inventory catalog, **Mercado Pago** payments, deployment, and the maintenance it still gets.
- Authentication, session security and **RBAC** that keep the staff side separate from the customer side, hardened against the **OWASP Top 10**: parameterized queries, server-side validation, CSRF on state-changing routes, least-privilege database roles.

PairCode | *Live Demo*

Node.js | **Postgres** | **WebSockets**

Real-time collaborative workspace with an identity layer and realtime transport I wrote myself.

- I wrote the auth layer instead of importing one: **EdDSA**-signed JWTs, rotating refresh tokens that detect reuse, **Argon2id** hashing and CSRF protection. The custom **WebSocket server** consumes a single-use ticket on handshake and authorizes every event on the server, so a client cannot promote itself by editing a payload.

Work Experience

Tambora

Jul 2025 – Sep 2025

Frontend Developer (Contract)

Remote

- Read through a legacy jQuery codebase and migrated its business-critical modules to **React**. Bundle size dropped **40%** on the rebuilt surfaces, and the ad hoc DOM string injection went with it.
- Replaced manual SSH deploys with **Azure CI/CD** pipelines and put test gates in front of every release. A deploy went from over **two hours to under fifteen minutes**.

Technical Skills

AI engineering: spec-driven development with coding agents, Claude Code, Anthropic SDK, Vercel AI SDK, MCP servers, RAG pipelines, structured outputs, agent evals and benchmarks, prompt-injection defense.

Languages: TypeScript, JavaScript, Python, C, SQL, Bash.

Backend: Node.js, Express, Next.js (App Router, RSC, Server Actions), FastAPI, REST, WebSockets, Prisma, PostgreSQL, MongoDB, Redis.

Frontend: React 19, Next.js, Tailwind CSS, React Flow, Liveblocks, Astro, Zod, Jest.

Architecture & security: design patterns, Atomic Design, ts-morph static analysis, threat modeling (STRIDE), OWASP Top 10 (Web + LLM), OAuth 2.0 / OIDC, JWT, RBAC, secure code review.

Cloud & DevOps: AWS, Azure, Docker, CI/CD (Azure Pipelines, GitHub Actions), Linux, Git.

Education & Certifications

Universidad Tecnológica de Pereira, Pereira, Colombia. Systems & Computing Engineering, in progress (Feb 2022 to present).

EliteStack Bootcamp, Pereira, Colombia. Full-Stack Development (Jun 2024 to Jul 2024).

Certifications: Google Professional Cybersecurity Certificate, PortSwigger Web Security Academy. In progress: BSCP, CompTIA Security+.